SEQUENCES, CONTAINERS, OBJECTS, & LINKED LISTS

COMPUTER SCIENCE MENTORS 61A

September 28, 2026 – October 2, 2026

1 Sequences

Sequence Overview

Sequences are ordered data structures that have length and support element selection. Here are some common types of sequences you'll be dealing with in this class:

- Lists: `[1, [2], 'a', lambda x: 5]`
- Tuples: `(1, (2,), 'a', lambda x: 5)`
- Strings: `'Hello World!'`

While each type of sequence is different, they all share a common interface for manipulating and accessing their data:

- **Item selection**: Use square brackets to select an element at an index:

```
(3, 1, 2)[0]   # returns 3
"Hello"[-1]    # returns "o"
```

- **Length**: The built-in **`len`** function returns the length of a sequence:

```
len((1, 2))   # returns 2
```

- **Concatenation**: Sequences can be concatenated with the + operator, which returns a *new* sequence:

```
[1, 2] + [3, 4]   # returns [1, 2, 3, 4]
```

- **Membership**: The **`in`** operator tests for sequence membership:

```
1 in (1, 2, 3)           # returns True
5 not in (1, 2, 3)      # returns True
"apple" in "snapple"    # returns True
```

  Membership in Strings vs. Lists and Tuples: As a short aside, while the **`in`** operator works the same for lists and tuples, checking if an element is contained within the list/tuple container, the **`in`** operator instead for strings checks for direct substrings rather than the existence of distinct elements within the string.

- **Looping**: Sequences can be looped through with **`for`** loops:

```
>>> for x in [1, 2, 3]:
...     print(x)
1
2
3
```


Provided by CSM 61A contributors under CC BY 4.0/3-clause BSD.

- **Aggregation**: Common built-in functions—including **`sum`**, **`min`**, and **`max`**—can take sequences and aggregate them into a single value:

  ```
  max((3, 4, 5))  # returns 5
  ```

- **Slicing**: Slicing is a way to create a copy of all or part of a sequence. The general syntax for slicing a sequence `seq` is as follows:

  ```
  seq[<start index>:<end index>:<step size>]
  ```

  This evaluates to a new sequence that includes every element starting at `<start index>` and up to and *excluding* `<end index>` in `seq`, taking steps of size `<step size>`.
  If we do not supply `<start index>` or `<end index>`, it will start at the beginning of the sequence and include every element up to and including the end of the sequence.

  ```
  >>> lst = [1, 2, 3, 4, 5]
  >>> lst[2:]
  [3, 4, 5]
  >>> lst[:3]
  [1, 2, 3]
  >>> lst[::-1]
  [5, 4, 3, 2, 1]
  >>> lst[1::2]
  [2, 4]
  ```

List comprehensions create a new list by processing an iterable, such as a list, tuple, string, or range. The syntax for a list comprehension is

```
[<expression> for <element> in <sequence> if <condition>]
```

We could equivalently write the following:

```
lst = []
for <element> in <sequence>:
    if <condition>:
        lst = lst + [<expression>]
```

The **`if`** `<condition>` filter statement is optional. The following list comprehension doubles each odd element of [1, 2, 3, 4]:

```
>>> [i * 2 for i in [1, 2, 3, 4] if i % 2 != 0]
[2, 6]
```

Equivalent in **`for`** loop syntax:

```
lst = []
for i in [1, 2, 3, 4]:
    if i % 2 != 0:
        lst = lst + [i * 2]
```

Sequence Practice

1. What would Python display? Run the statements in order. For nested expressions, work from the innermost expression outward.

```
>>> a = [1, 2, 3]
>>> a
```

```
>>> a[2]
```

```
>>> a[-1]
```

```
>>> a[1:]
```

```
>>> a[:2]
```

```
>>> a + [4, 5]
```

```
>>> a
```

```
>>> nested = [1, 2, 3, 4, [5, 6]]
>>> nested[4]
```

```
>>> nested[4][0]
```

```
>>> nested[3:5]
```

2. **Tuples.** What would Python display? Run these statements in order. If a statement causes an error, identify the error and continue with the next statement. For `(3,)` and `(3),` also name the type.

Creating tuples

```
>>> t: tuple[int, str, float, bool, None] = (
...     3, "csm", 2.5, True, None
... )
>>> t
```

```
>>> 4, 5, 6, 7
```

```
>>> (3,)
```

```
>>> (3)
```

```
>>> ()
>>> len(())
```

Slicing and converting

```
>>> t[1:4]
```

```
>>> t[::-1]
```

```
>>> list(t)
```

```
>>> list("csm")
>>> list(())
```

Combining tuples

```
>>> (4, 5) + (6, 7)



>>> (3,) * 3
>>> (3) * 3
```

Dictionary keys

```
>>> locations = {(4, 5): "CSM"}
>>> locations[(4, 5)]



>>> locations[[4, 5]]



>>> locations[(4, [5])]
```

3. Write a function `duplicate_list`, which takes in a list of positive integers and returns a new list with each element `x` in the original list duplicated `x` times.

```
def duplicate_list(lst: list[int]) -> list[int]:
    """
    >>> duplicate_list([1, 2, 3])
    [1, 2, 2, 3, 3, 3]
    >>> duplicate_list([5])
    [5, 5, 5, 5, 5]
    """
    ____________________________________

    for ____________________________________:

        for ____________________________________:

            ____________________________________________

    ____________________________________
```

4. Write a function that takes in a list `nums` and returns a new list with only the primes from `nums`. Assume that `is_prime(n)` is defined. You may use a **`while`** loop, a **`for`** loop, or a list comprehension.

```
def all_primes(nums: list[int]) -> list[int]:
```

5. Fill in the functions below according to the doctests.

```
def gen_list(n: int) -> list[list[int]]:
    """
    Returns a nested list structure of n elements where the
    ith element is a list from 0 (inclusive) to i (exclusive).
    >>> gen_list(3)
    [[0], [0, 1], [0, 1, 2]]
    >>> gen_list(5)
    [[0], [0, 1], [0, 1, 2], [0, 1, 2, 3], [0, 1, 2, 3, 4]]
    """
    return _____________________________________________________
```

For an additional challenge, try out the following:

```
def gen_increasing(n: int) -> list[list[int]]:
    """
    Returns a nested list structure of n elements where the
    ith element of each list is one more than the previous
    element (even if the previous is in a prior sublist).
    >>> gen_increasing(3)
    [[0], [1, 2], [3, 4, 5]]
    >>> gen_increasing(5)
    [[0], [1, 2], [3, 4, 5], [6, 7, 8, 9], [10, 11, 12, 13,
    14]]
    """
    return _____________________________________________________
```

Hint: You can sum ranges. E.g. `sum(range(3))` gives us $0 + 1 + 2 = 3$.

6. Write `count_t`, which returns the number of lowercase `"t"` characters in `word`. Then use it to construct a dictionary mapping each word in `words` to its count.

```
def count_t(word: str) -> int:
    """
    >>> count_t("tatter")
    3
    >>> count_t("tree")
    1
    >>> count_t("")
    0
    """
    ____________________________________

    for ____________________________________:

        if ____________________________________:

            ________________________________________

    ____________________________________
```

Complete the dictionary comprehension. Its result should be `{'tatter': 3, 'tree': 1}`.

```
words = ["tatter", "tree"]
counts: dict[str, int] = {
    _____________: _____________ for word in words
}
```

7. Complete `snapshot`. It takes a function from integers to integers and a list of integer inputs. Return a dictionary whose keys are those inputs and whose values are the corresponding outputs of the function. Use a dictionary comprehension.

```
from collections.abc import Callable

def snapshot(
    f: Callable[[int], int], snap_inputs: list[int]
) -> dict[int, int]:
    """
    >>> snapshot(lambda x: x**2, [1, 2, 3])
    {1: 1, 2: 4, 3: 9}
    >>> snapshot(lambda x: x + 1, [])
    {}
    """
    return {
        _____________: _____________
        for ____________________________________
    }
```

8. (a) What is an **Abstract Data Type**?

   (b) What is the relationship between a class and an ADT?

9. **Session records.** What would Python display after executing the following definitions? Read the field annotations before tracing.

```
from dataclasses import dataclass

@dataclass
class Session:
    topic: str
    room: int
    minutes: int = 60

morning = Session("Recursion", 310)
afternoon = Session("Sequences", 205, 45)
```

```
>>> morning.topic
```

```
>>> morning.minutes
```

```
>>> afternoon.room


>>> afternoon.minutes


>>> morning.minutes + afternoon.minutes


>>> morning


>>> Session(room=220, topic="Linked lists").topic
```

4 Linked Lists

For the following questions, use this dataclass-based linked list. A `LinkedList[T]` is either a `Link[T]` or the empty tuple `()`. The type variable `T` describes the element type.

```
from __future__ import annotations
from dataclasses import dataclass

type LinkedList[T] = Link[T] | tuple[()]

@dataclass
class Link[T]:
    first: T
    rest: LinkedList[T] = ()

    def __str__(self) -> str:
        return format_link(self)

def format_link[T](s: Link[T]) -> str:
    string = '(' + str(s.first)
    remaining = s.rest
    while isinstance(remaining, Link):
        string += ' ' + str(remaining.first)
        remaining = remaining.rest
    assert remaining == (), 'Not a valid linked list'
    return string + ')'
```

Construct a list with `Link(1, Link(2))`. Read its elements with `first` and `rest`. At the end, `rest` is `()`, which has no `first` or `rest` attributes. **`print`** uses parentheses around the elements; entering a `Link` at the prompt shows the dataclass representation with field names.

10. What would Python display? Run these statements in order. If a statement causes an error, identify the error and continue.

```
>>> a = Link(1, Link(2, Link(3)))
>>> a.first
```

```
>>> a.rest.first
```

```
>>> a.rest.rest.first
```

```
>>> a.rest.rest.rest
```

```
>>> isinstance(a.rest, Link)
```

```
>>> isinstance(a.rest.rest.rest, Link)
```

```
>>> a.rest.rest.rest.first
```

```
>>> a.rest.rest
```

```
>>> Link(1, Link(2))
```

```
>>> print(a)
```

11. Write a function `linkify` that, given a list `lst`, returns a linked list with the same values. You may assume that there will be no nested lists.

    Try writing this function both recursively and iteratively.

```
def linkify_rec[T](lst: list[T]) -> LinkedList[T]:
    """
    >>> lst = [0, 1, 2, 3]
    >>> print(linkify_rec(lst))
    (0 1 2 3)
    >>> linkify_rec([])
    ()
    """
    if ________________________________________:

        ____________________________________

    else:

        ____________________________________


def linkify_iter[T](lst: list[T]) -> LinkedList[T]:

    ____________________________________________

    for ________________________________________:

        ____________________________________

    ____________________________________________
```

12. Write a function `combine_two`, which takes in a linked list of integers `lnk` and a two-argument function `fn`. It returns a new linked list where every two elements of `lnk` have been combined using `fn`.

```
from collections.abc import Callable
def combine_two(
    lnk: LinkedList[int], fn: Callable[[int, int], int]
) -> LinkedList[int]:
    """
    >>> from operator import add, mul
    >>> lnk1 = Link(1, Link(2, Link(3, Link(4))))
    >>> print(combine_two(lnk1, add))
    (3 7)
    >>> lnk2 = Link(2, Link(4, Link(6)))
    >>> print(combine_two(lnk2, mul))
    (8 6)
    >>> combine_two((), add)
    ()
    """
    if _____________________________________________:

        return _____________________________________
    elif _____________________________________________:

        return _____________________________________
    combined = _____________________________________

    return _____________________________________________
```

5 Optional Extension: Tree-Recursive Counting

13. Gabe's Donut shop has an unlimited supply of f different flavors of donuts. Adit wants to buy a box containing d donuts. Complete the skeleton for the function `donut`, which determines the number of possible ways there are for Adit to select his d donuts from the f flavors. You may assume that `d` and `f` are non-negative integers.

    Hint: Does order matter?

```
def donut(d: int, f: int) -> int:
    """
    >>> donut(12, 1)
    1
    >>> donut(12, 2)
    13
    >>> donut(12, 12)
    1352078
    >>> donut(0, 0)
    1
    """
    if ____________________:

        return ____________________

    if ____________________:

        return ____________________

    return ________________________________________________
```